Análisis de resultados — Sprint 4

Qué se probó, qué se encontró y qué significa para el negocio.

1. Qué se probó

Área	Casos	Qué protege
Acceso: login, sesión única, permisos por rol	01–09, 12, 14, 15	Que solo entre quien debe, y que cada uno vea solo lo suyo
Disponibilidad del cadete	10, 11, 16	Que solo reciban viajes los cadetes que pueden tomarlos, con la app al día
Ciclo de vida de un pedido	13, 17–20, 22	El servicio que se vende: del alta a la entrega
Controles antifraude	21, 22	Que no se cobre ni se pague un viaje que no se hizo

Resultado: 22 de 22 casos aprobados en 30 corridas sobre 3 bases nuevas (3.875 aserciones, 0 fallidas).

2. Hallazgos

Cada hallazgo tiene **qué pasó**, **qué significa para el negocio** y **qué recomendamos**.

H1 — El mock daba verde donde la API real falla · Proceso de prueba · Severidad alta

Qué pasó. En el Sprint 3, el pipeline corría la suite contra un servidor simulado escrito por el equipo, y daba 100 % de aprobados. Contra la API real, con una base nueva, la misma suite falló en las 30 corridas. El paso Z1 devolvía `400 DATO_INVALIDO: "Tu versión de la app es vieja"`. El backend había incorporado esa regla (versión mínima de la app para ponerse Libre) y el mock nunca se enteró.

Qué significa para el negocio. Durante un sprint el equipo creyó que todo andaba. Si esa suite hubiera sido la "puerta" antes de un despliegue, habría aprobado una versión sin verificarla contra el sistema real. Un verde falso es peor que no tener pruebas, porque da confianza que no está justificada.

Qué recomendamos (y aplicamos). El pipeline corre contra la API real, levantada desde cero en cada ejecución. El mock se eliminó.

H2 — La versión de la app que se controla depende del último login · Sistema / prueba · Severidad media

Qué pasó. Para dejar a un cadete Libre, la API controla la versión de la app. Si el pedido no trae el header `X-App-Version`, usa la versión con la que el cadete inició sesión la última vez. Si nunca la mandó, toma 0. El mismo pedido da 200 o 400 según quién y cómo haya entrado antes.

Qué significa para el negocio. Es una decisión de diseño válida: las apps anteriores a la versión 3 no mandan el header, y así siguen funcionando. Pero introduce **estado oculto**: el resultado no depende solo del pedido. Para soporte, "a mí me deja ponerme Libre y a él no" puede deberse a eso.

Qué recomendamos. En la suite, mandar siempre el header (aplicado). El caso 16 manda versión 0 a propósito, para verificar el rechazo sin depender del historial. Al equipo de desarrollo: documentar esta regla en el contrato de la API.

H3 — Los datos de demostración se reparten al azar · Datos de prueba · Severidad media

Qué pasó. La carga de demostración asigna algunos pedidos finalizados y un reclamo abierto "al primer cadete que devuelve la base". Ese orden depende de un identificador aleatorio (UUID). Con cada base nueva le puede tocar a un cadete distinto. Consecuencias: - El caso 13 (que tomaba un pedido del historial del cadete) falló en 1 de 6 bases nuevas sin que hubiera un error del sistema. - Cuando el reclamo abierto le toca al cadete de prueba, no se le puede asignar ningún pedido. La API responde, con razón, `409: "El cadete tiene un incidente abierto por un reclamo"`.

Qué significa para el negocio. Las pruebas que fallan "a veces" hacen que el equipo deje de creerle al pipeline y termine ignorando los rojos, incluso los reales.

Qué recomendamos (y aplicamos). La suite crea el pedido que usa y cierra los incidentes abiertos del cadete antes del flujo (paso P6), como lo haría un admin. Para desarrollo: que la carga demo elija los cadetes de forma determinística.

H4 — El job contra la API real del Sprint 3 nunca podía funcionar · Pipeline · Severidad media

Qué pasó. El job `api-real` buscaba el archivo `*SNAPSHOT.jar`, pero el backend genera `backend-0.1.0.jar`. Además, solo se activaba si alguien definía una variable del repositorio, y nunca se definió.

Qué significa para el negocio. "Tenemos CI contra la API real" era cierto en el papel. Ese tipo de brecha entre lo que se informa y lo que pasa es la que más daña la confianza de un cliente.

Qué recomendamos (y aplicamos). El job se reescribió, se activa por defecto y se verificó en local con base nueva. Aun así, la primera corrida en GitHub falló por otro detalle que en local no aparecía: `actions/checkout` no acepta el hash corto del commit del backend. Se corrigió con el hash completo y las corridas siguientes dieron 22 de 22. La lección se confirma: hasta no verlo en verde en el lugar donde va a correr, no está probado.

H5 — Cualquiera que conozca el DNI de un cadete puede bloquearle la cuenta · Seguridad · Severidad alta

Qué pasó. Al correr la suite contra la copia del instituto con contraseñas equivocadas, la API respondió `429 DEMASIADAS_SOLICITUDES: "Cuenta bloqueada temporalmente por intentos fallidos. Probá de nuevo en 14 minuto(s)"`. Revisamos el código: después de `max_intentos_login` (5) intentos fallidos, la **cuenta** queda bloqueada 15 minutos. El bloqueo es por cuenta, no por dirección IP, y el usuario del cadete es su **DNI**.

Qué significa para el negocio. El bloqueo protege contra quien intenta adivinar contraseñas, que es lo correcto. El efecto secundario es que alguien que conozca el DNI de un cadete puede dejarlo sin poder entrar, una y otra vez, con solo 5 intentos cada 15 minutos. Un cadete que no puede iniciar sesión no trabaja: es plata perdida para él y para la empresa, y un reclamo seguro.

Qué recomendamos. Reportarlo al equipo de desarrollo. Algunas opciones: - Contar los intentos por cuenta **y** por IP, y bloquear por la combinación. - Pedir un captcha después de 3 fallos en lugar de bloquear. - Avisarle al cadete y al admin cuando su cuenta se bloquea.

Cuando se corrija, sumar un caso de regresión. Mientras tanto, la suite nunca se corre en bucle contra un servidor real, y las contraseñas viven en secretos.

H6 — Asignar un pedido anulado da un mensaje que no corresponde · Usabilidad · Severidad baja

Qué pasó. En la prueba exploratoria, asignar un pedido ya **anulado** devuelve `409 CONFLICTO` (correcto), pero con el mensaje *"El pedido ya tiene una asignación en curso"*. Anular dos veces devuelve el mensaje correcto: *"El pedido ya está cerrado"*.

Qué significa para el negocio. El operador lee que el pedido "tiene una asignación en curso", va a buscar al cadete, no lo encuentra y llama a soporte. Es tiempo perdido, aunque el sistema no hizo nada mal.

Qué recomendamos. Que la API verifique primero si el pedido está cerrado, y use el mismo mensaje en los dos casos.

H8 — El entorno compartido no estaba en condiciones de probar · Entorno de prueba · Severidad media

Qué pasó. Al preparar el job contra la copia del instituto, ningún cadete podía iniciar sesión. Ninguno tenía la cuota semanal habilitada (el backend deshabilita a todos los cadetes de pago semanal cada lunes a las 0:00), y al cadete de prueba le habían reenviado la contraseña y la temporal había vencido sin usarse.

Qué significa para el negocio. Un entorno de prueba que nadie mantiene termina siendo inutilizable justo cuando hace falta, como el día de una demo. Y la regla del lunes, correcta para el negocio, rompe cualquier prueba automática que no la tenga en cuenta.

Qué recomendamos (y aplicamos). Se dejaron los cadetes del instituto con las contraseñas de la carga demo, y la suite suma la precondición P1b: si el cadete de prueba no tiene la cuota paga, la habilita como lo haría el admin. Con eso, la corrida contra el instituto dio 22 de 22.

H7 — Las reglas antifraude y de disponibilidad funcionan · Verificación

Regla	Caso	Resultado
Una app vieja no puede ponerse Libre	16	`400 DATO_INVALIDO`
Solo se asigna a un cadete libre y sin reclamos abiertos	18 (y la exploración)	`409 CONFLICTO` si no se cumple
Entre Retirado y Finalizar tienen que pasar al menos 10 minutos	21	`400 DATO_INVALIDO` con el tiempo que falta
El admin puede cerrar a mano, pero solo con motivo	22 (y la exploración)	`200 FINALIZADO` con motivo; `400 VALIDACION` sin él
Finalizar dos veces (la app reintenta sin señal) no da error ni duplica	Exploración	`200` con el pedido ya finalizado

Qué significa para el negocio. Son los controles que evitan cobrar o pagar viajes que no se hicieron y ofrecer viajes a quien no puede tomarlos. Quedan verificados en cada push y todos los días.

3. Resultados por corrida (Sprint 4, base nueva cada vez)

Base	Corridas	Requests	Aserciones	Fallidas	Promedio de respuesta	Cobertura de líneas
1	10	281	1.291	0	47 ms	32,2 %
2	10	281	1.291	0	42 ms	32,2 %
3	10	283	1.293	0	44 ms	32,0 %

Las diferencias en requests y aserciones de la base 3 salen del paso P6: cuando el cadete de prueba tiene un incidente abierto, se agrega un pedido para cerrarlo y su verificación.

4. Conclusión para el cliente

- La suite es **estable**: 30 de 30 corridas en verde con base nueva. Antes: 0 de 30.
- La suite es **confiable**: corre contra la API real, no contra una imitación.
- La suite es **útil para el negocio**: además del acceso, verifica las reglas que protegen la plata (antifraude) y la operación (disponibilidad).
- La suite ya encontró algo importante: un riesgo de seguridad (H5) que conviene corregir antes de que alguien lo use.